



**FUNDAÇÃO EDSON QUEIROZ
UNIVERSIDADE DE FORTALEZA
CENTRO DE CIÊNCIAS TECNOLÓGICAS
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO**

**Matheus Diógenes Amorim – 2310277
Matheus Holanda de Sá – 2320306**

ANÁLISE COMPARATIVA DE ALGORITMOS COM USO DE PARALELISMO

**Computação Concorrente e Paralela
SerialCPU, ParallelCPU e ParallelGPU**

**Fortaleza – CE
2026**

Sumário

1	Resumo	3
2	Introdução	3
3	Objetivos	3
4	Metodologia	4
4.1	Entradas	4
4.2	Normalização	4
4.3	Execuções	4
4.4	Organização do CSV	5
5	Implementação	5
5.1	SerialCPU	6
5.2	ParallelCPU	6
5.3	ParallelGPU	6
6	Ambiente e Dependências	6
7	Como Compilar e Executar	7
7.1	Compilar	7
7.2	Executar benchmark completo	7
7.3	Executar contagem manual	7
8	Resultados	7
9	Gráficos Comparativos	8
9.1	Tempo médio por método	9
9.2	Tempo médio por arquivo	10
9.3	Speedup CPU	11
10	Discussão dos Resultados	11
11	Conclusão	12
12	Referências	12
13	Anexos	13
13.1	Códigos principais da implementação	13
13.2	Arquivos de saída	13

1 Resumo

Este projeto compara três abordagens para a contagem de ocorrências de uma palavra em arquivos de texto: execução serial na CPU, execução paralela na CPU com threads e execução paralela na GPU com OpenCL.

Os testes foram executados com três textos de tamanhos diferentes, três repetições por configuração e variação da quantidade de threads no método paralelo em CPU. Os resultados foram registrados em arquivo CSV e apresentados em gráficos comparativos para analisar o tempo médio, o impacto do tamanho da entrada, o speedup da CPU paralela e o custo de uso da GPU.

2 Introdução

A busca por uma palavra em um texto é uma operação simples, mas adequada para observar diferenças entre execução sequencial e paralela.

Em entradas maiores, dividir o trabalho entre vários núcleos de CPU pode reduzir o tempo total de processamento. Já a GPU oferece grande quantidade de unidades de processamento, mas também possui custos de inicialização, transferência de memória e compilação do kernel OpenCL.

O trabalho implementa os três métodos solicitados no enunciado: **SerialCPU**, **ParallelCPU** e **ParallelGPU**. Todos os métodos usam a mesma etapa de normalização para garantir que a comparação seja feita com a mesma regra de contagem.

Tabela 1: Métodos implementados

Método	Descrição
SerialCPU	Percorre o texto normalizado sequencialmente em uma única thread.
ParallelCPU	Divide o texto em partes e processa as partes em paralelo usando <code>ExecutorService</code> .
ParallelGPU	Envia o texto e a palavra para a GPU e executa um kernel OpenCL com JOCL.

3 Objetivos

- Implementar um contador de palavras serial em Java.
- Implementar um contador de palavras paralelo em CPU com pool de threads.
- Implementar um contador de palavras paralelo em GPU com OpenCL.
- Executar testes com textos de tamanhos diferentes.
- Realizar pelo menos três repetições por abordagem.
- Variar a quantidade de threads no método **ParallelCPU**.

- Registrar os resultados em CSV.
- Gerar gráficos comparativos.
- Produzir um relatório com os resultados e o link do projeto no GitHub.

4 Metodologia

4.1 Entradas

Foram usados os textos disponíveis na pasta **Amostras/**. A Tabela 2 apresenta os arquivos, as palavras buscadas e a quantidade de ocorrências encontradas por todos os métodos.

Tabela 2: Arquivos de entrada e palavras buscadas

Arquivo	Palavra buscada	Ocorrências encontradas
DonQuixote-388208.txt	que	21618
Dracula-165307.txt	the	8104
MobyDick-217452.txt	the	14727

4.2 Normalização

Antes da contagem, os textos e a palavra buscada passam pela classe `TextNormalizer`, que aplica as seguintes regras:

- conversão para letras minúsculas;
- remoção de acentos;
- substituição de pontuação e caracteres especiais por espaços;
- padronização de espaços;
- validação para garantir que a busca seja feita por apenas uma palavra.

Com isso, as três implementações contam a mesma palavra normalizada e evitam divergências causadas por pontuação, maiúsculas ou acentos.

4.3 Execuções

Para cada arquivo, foram executadas as configurações apresentadas na Tabela 3.

Cada configuração foi executada três vezes. O tempo foi medido em milissegundos com `System.nanoTime()` e registrado em `resultados/resultados.csv`.

Tabela 3: Configurações executadas

Abordagem	Configuração
SerialCPU	1 thread
ParallelCPU	2 threads
ParallelCPU	4 threads
ParallelCPU	8 threads
ParallelGPU	GPU via OpenCL

4.4 Organização do CSV

O arquivo CSV contém os dados brutos das medições. Cada linha representa uma execução individual de uma configuração.

A coluna `repeticao` indica se aquela linha corresponde à primeira, segunda ou terceira execução do mesmo teste. Assim, o mesmo arquivo, palavra, método e quantidade de threads aparecem três vezes no CSV, uma vez para cada repetição.

O formato do CSV é:

```
arquivo,palavra,metodo,threads,repeticao,ocorrencias,tempo_ms
```

Exemplo de três repetições do mesmo teste:

```
DonQuixote-388208.txt,que,SerialCPU,1,1,21618,6.530
DonQuixote-388208.txt,que,SerialCPU,1,2,21618,4.269
DonQuixote-388208.txt,que,SerialCPU,1,3,21618,3.750
```

Como foram usados três arquivos, cinco configurações por arquivo e três repetições por configuração, o CSV possui 45 linhas de dados, além da linha de cabeçalho. O terminal mostra um resumo com as médias, enquanto o CSV preserva todas as medições individuais.

5 Implementação

A estrutura principal do projeto é:

```
|-- Amostras/
|-- openc1/
| '-- word_count.cl
|-- resultados/
| |-- resultados.csv
| '-- graficos/
|-- src/
| |-- Main.java
| |-- BenchmarkRunner.java
| |-- BenchmarkResult.java
| |-- ChartGenerator.java
| |-- CsvWriter.java
| |-- ParallelCPUCounter.java
| |-- ParallelGPUCounter.java
| |-- SerialCPUCounter.java
```

```
| |-- TextNormalizer.java
| '-- WordCounter.java
'-- jocl-2.0.4.jar
```

5.1 SerialCPU

O método `SerialCPU` percorre o texto normalizado palavra por palavra. Para cada palavra encontrada no texto, compara o tamanho e os caracteres com a palavra buscada. Como a execução é sequencial, o custo cresce de forma direta com o tamanho do texto.

5.2 ParallelCPU

O método `ParallelCPU` usa `ExecutorService` para criar um pool de threads. O texto é dividido em faixas, cada thread conta as ocorrências dentro da sua parte e, ao final, os resultados parciais são somados.

Um cuidado importante está no tratamento das bordas das faixas. Se uma faixa começa no meio de uma palavra, a implementação avança até o próximo espaço para evitar contagem duplicada ou parcial.

5.3 ParallelGPU

O método `ParallelGPU` usa JOCL para acessar OpenCL. O texto normalizado, a palavra buscada e o vetor de resultados são enviados para a GPU.

O kernel `count_words` atribui uma posição do texto para cada work-item e marca 1 quando encontra uma ocorrência válida da palavra. Ao final, o programa lê o vetor de resultados da GPU e soma as ocorrências na CPU.

6 Ambiente e Dependências

O projeto foi desenvolvido em Java e usa JOCL para executar o método em GPU.

O arquivo `jocl-2.0.4.jar` está na raiz do projeto e deve permanecer nesse local, pois os comandos de compilação e execução usam esse caminho.

Tabela 4: Dependências do projeto

Dependência	Finalidade
JDK 21 ou compatível	Compilar e executar o projeto Java.
<code>jocl-2.0.4.jar</code>	Biblioteca Java para acesso ao OpenCL.
Driver OpenCL	Necessário para executar <code>ParallelGPU</code> .
GPU compatível com OpenCL	Dispositivo usado pelo método <code>ParallelGPU</code> .

Se o método `ParallelGPU` falhar, é necessário verificar se o driver OpenCL está instalado e se a biblioteca nativa `libOpenCL.so` está disponível no sistema.

7 Como Compilar e Executar

7.1 Compilar

```
javac -cp jocl-2.0.4.jar -d build src/*.java
```

7.2 Executar benchmark completo

```
java -cp build:jocl-2.0.4.jar Main benchmark
```

Esse comando executa os testes padrão, gera o CSV em `resultados/resultados.csv` e atualiza os gráficos em `resultados/graficos/`.

7.3 Executar contagem manual

```
java -cp build:jocl-2.0.4.jar Main contar Amostras/Dracula-165307.txt the
```

Também é possível informar a quantidade de threads do `ParallelCPU`:

```
java -cp build:jocl-2.0.4.jar Main contar Amostras/Dracula-165307.txt vampire 4
```

8 Resultados

O arquivo completo de resultados está em `resultados/resultados.csv`. A Tabela 5 apresenta as médias das três repetições de cada configuração.

As ocorrências foram iguais em todos os métodos para cada arquivo, confirmando a consistência das implementações.

Tabela 5: Médias das três repetições por configuração

Arquivo	Palavra	Método	Threads	Média (ms)	Ocorrências
DonQuixote-388208.txt	que	SerialCPU	1	4.850	21618
DonQuixote-388208.txt	que	ParallelCPU	2	9.251	21618
DonQuixote-388208.txt	que	ParallelCPU	4	1.990	21618
DonQuixote-388208.txt	que	ParallelCPU	8	1.610	21618
DonQuixote-388208.txt	que	ParallelGPU	0	71.303	21618
Dracula-165307.txt	the	SerialCPU	1	1.747	8104
Dracula-165307.txt	the	ParallelCPU	2	1.235	8104
Dracula-165307.txt	the	ParallelCPU	4	0.918	8104
Dracula-165307.txt	the	ParallelCPU	8	1.067	8104
Dracula-165307.txt	the	ParallelGPU	0	52.636	8104
MobyDick-217452.txt	the	SerialCPU	1	2.524	14727
MobyDick-217452.txt	the	ParallelCPU	2	1.639	14727
MobyDick-217452.txt	the	ParallelCPU	4	1.193	14727
MobyDick-217452.txt	the	ParallelCPU	8	1.294	14727
MobyDick-217452.txt	the	ParallelGPU	0	50.005	14727

Tabela 6: Médias gerais por método

Método	Média geral (ms)
SerialCPU	3.040
ParallelCPU 2 threads	4.042
ParallelCPU 4 threads	1.367
ParallelCPU 8 threads	1.324
ParallelGPU	57.981

O speedup foi calculado pela razão entre o tempo médio do **SerialCPU** e o tempo médio do **ParallelCPU** para cada arquivo. Em seguida, foi calculada a média dos speedups dos três arquivos.

Tabela 7: Speedup médio do ParallelCPU

Configuração	Speedup médio
ParallelCPU 2 threads	1.159x
ParallelCPU 4 threads	2.152x
ParallelCPU 8 threads	2.200x

9 Gráficos Comparativos

Os gráficos a seguir foram gerados pelo projeto a partir dos resultados do benchmark.

9.1 Tempo médio por método



Figura 1: Tempo médio por método.

9.2 Tempo médio por arquivo



Figura 2: Tempo médio por arquivo, considerando todas as configurações executadas.

9.3 Speedup CPU

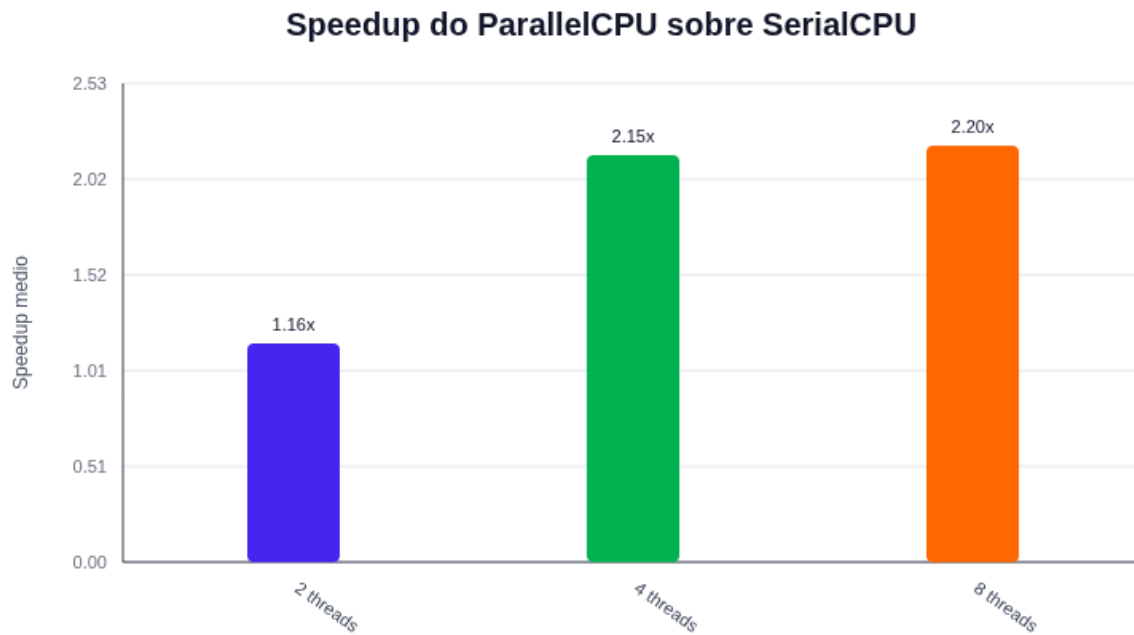


Figura 3: Speedup médio do ParallelCPU em relação ao SerialCPU.

10 Discussão dos Resultados

Os resultados mostram que o ParallelCPU apresentou o melhor desempenho geral quando configurado com 4 ou 8 threads. Em `DonQuixote-388208.txt`, o tempo médio caiu de 4.850 ms no método serial para 1.610 ms com 8 threads, indicando ganho relevante com a divisão do trabalho entre núcleos de CPU.

Com 2 threads, o ganho não foi uniforme. No arquivo `DonQuixote-388208.txt`, o ParallelCPU com 2 threads ficou mais lento que o método serial.

Isso pode ocorrer por causa do custo de criação e gerenciamento das threads, além da divisão do texto em partes. Para entradas pequenas ou configurações pouco favoráveis, o overhead do paralelismo pode superar o ganho obtido.

O ParallelCPU com 4 threads apresentou bom equilíbrio, sendo mais rápido que o serial nos três arquivos.

Com 8 threads, o desempenho também foi bom, mas, em `Dracula-165307.txt` e `MobyDick-217452.txt`, o tempo médio ficou maior que com 4 threads. Isso indica que aumentar a quantidade de threads nem sempre melhora o resultado, pois há custos de coordenação e disputa por recursos da CPU.

O método ParallelGPU retornou as mesmas contagens dos métodos de CPU, mas apresentou tempos maiores nas amostras testadas.

A principal explicação é que o custo de inicializar OpenCL, compilar o kernel, criar buffers, transferir dados para a GPU e ler o resultado de volta foi maior que o ganho obtido

na contagem. Para volumes de dados muito maiores ou para kernels mais complexos, a GPU poderia se tornar mais competitiva.

Também foi observado que todas as abordagens retornaram exatamente a mesma quantidade de ocorrências em cada arquivo. Isso confirma que a normalização e a regra de contagem foram aplicadas de maneira consistente entre CPU serial, CPU paralela e GPU.

11 Conclusão

O trabalho demonstra que o paralelismo em CPU pode reduzir o tempo de execução da contagem de palavras, especialmente quando são usadas 4 ou 8 threads. No conjunto de testes realizado, o `ParallelCPU` foi a abordagem mais eficiente.

A GPU funcionou corretamente e produziu as mesmas contagens, mas não foi a alternativa mais rápida para essas entradas.

O resultado evidencia que paralelizar uma tarefa não garante automaticamente melhor desempenho: é necessário considerar tamanho da entrada, custo de comunicação, inicialização do ambiente paralelo e quantidade de trabalho executada por cada unidade de processamento.

Assim, para a contagem de palavras nos textos testados, a melhor escolha foi o processamento paralelo em CPU. A implementação em GPU permanece válida como comparação e como demonstração do uso de OpenCL com Java.

12 Referências

Referências

- [1] Oracle. *Java Documentation*. Disponível em: <https://docs.oracle.com/en/java/>.
- [2] Khronos Group. *OpenCL Overview*. Disponível em: <https://www.khronos.org/opencl/>.
- [3] JOCL. *Java Bindings for OpenCL*. Disponível em: <https://www.jocl.org/>.
- [4] Material da disciplina de Computação Concorrente e Paralela.
- [5] Enunciado da atividade: `analise_comparativa_algoritmos_paralelismo.pdf`.

13 Anexos

13.1 Códigos principais da implementação

Tabela 8: Arquivos principais do projeto

Arquivo	Descrição
src/Main.java	Ponto de entrada da aplicação e comandos de execução.
src/TextNormalizer.java	Normalização dos textos e palavras.
src/WordCounter.java	Interface comum dos contadores.
src/SerialCPUCounter.java	Implementação serial em CPU.
src/ParallelCPUCounter.java	Implementação paralela em CPU com threads.
src/ParallelGPUCounter.java	Implementação paralela em GPU com JOCL.
opencl/word_count.cl	Kernel OpenCL de contagem.
src/BenchmarkRunner.java	Execução dos benchmarks.
src/CsvWriter.java	Geração do CSV.
src/ChartGenerator.java	Geração dos gráficos SVG.

13.2 Arquivos de saída

Tabela 9: Arquivos de resultado

Arquivo	Descrição
resultados/resultados.csv	Resultados brutos dos testes.
resultados/graficos/tempo_por_metodo.svg	Gráfico de tempo médio por método.
resultados/graficos/tempo_por_arquivo.svg	Gráfico de tempo médio por arquivo.
resultados/graficos/speedup_cpu.svg	Gráfico de speedup do ParallelCPU.

14 Link do Projeto

Repositório no GitHub: https://github.com/MatheuzinDev/analise_comparativa_algoritmos_paralelismo